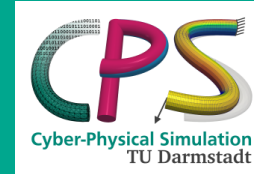


Tutorial Machine Learning in Solid Mechanics (Winter term 2025–2026)



TECHNISCHE
UNIVERSITÄT
DARMSTADT



Task 2: Hyperelasticity

Dominik K. Klein, M.Sc. , Jasper O. Schommartz, M.Sc. , Prof. Dr. Oliver Weeger
November 4, 2025

5 weeks from 12th Nov
→ First two weeks 1, 2
Next week onwards 1
Question.

In the first part of this task, see Sec. 1-Sec. 4, you will learn about (i) hyperelasticity, (ii) the generalisation of NNs, and (iii) construction principles for the *hard* fulfillment of mathematical conditions with NNs. In the second part of this task, see Sec. 5, you will learn about (iv) the trade-off between structure and flexibility and (v) the *soft* fulfillment of mathematical conditions with NNs.

1 Data preparation – Data from an analytical potential

In the first step, data for the calibration and testing of the NN models is to be prepared. For the calibration of material models, the corresponding datasets usually consist of stress-strain tuples which are received by physical experiments. However, in the following example data was generated *synthetically*, meaning by evaluating an analytical hyperelastic potential, and the resulting dataset

$$D = \{(\mathbf{F}_1, \mathbf{P}_1, W_1), (\mathbf{F}_2, \mathbf{P}_2, W_2), \dots\} \quad (1)$$

consists of triplets for the corresponding deformation gradient $\mathbf{F}$, the first Piola-Kirchhoff stress $\mathbf{P}(\mathbf{F})$ and the strain energy density $W(\mathbf{F})$. For the material models to be programmed, $\mathbf{F}$ will be the input, while $\mathbf{P}$ (and sometimes W) will be the output of the model.

1.1 Data import

Visit the GitHub repository [CPSHub/TutorialMLinSolidMechanics](#) and go to the folder “hyperelasticity/data”. We now consider the two folders which contain the calibration and test data, respectively. The data is stored in Voigt notation in text files, with the notation stored in a “Readme” file.

Implement a python script which imports $\mathbf{F}$, $\mathbf{P}$ and W from the text files. Reshape both $\mathbf{F}$ and $\mathbf{P}$ in such a way that you receive matrices, i.e.

$$\mathbf{F} = \begin{pmatrix} F_{11} & F_{12} & F_{13} \\ F_{21} & F_{22} & F_{23} \\ F_{31} & F_{32} & F_{33} \end{pmatrix}. \quad (2)$$

This data storage will facilitate the following tasks. Then, find a suitable visualization to examine the different load paths of the dataset (load path meaning e.g. uniaxial tension or the mixed test case).

Which physical experiments are represented in the load paths of the calibration dataset? What describes the test cases better: “interpolation” or “extrapolation”?

1.2 Analytical potential

Consider the transversely isotropic hyperelastic potential [8]

$$W(\mathbf{F}) = 8 I_1 + 10 J^2 - 56 \log(J) + 0.2 (I_4^2 + I_5^2) - 44 \quad (3)$$

with the invariants

$$I_1 = \text{tr } \mathbf{C}, \quad J = \det \mathbf{F}, \quad I_4 = \text{tr}(\mathbf{C} \mathbf{G}_{ii}), \quad I_5 = \text{tr}(\text{cof } \mathbf{C} \mathbf{G}_{ii}). \quad (4)$$

Here, $\mathbf{C} = \mathbf{F}^T \mathbf{F}$ denotes the right Cauchy-Green tensor and $\text{cof } \mathbf{C} = I_3 \mathbf{C}^{-T}$ its cofactor. Furthermore, the transversely isotropic structural tensor is given by

$$\mathbf{G}_{ii} = \begin{pmatrix} 4 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 0.5 \end{pmatrix}. \quad (5)$$

At first, implement all of above invariants in JAX. Check your implementation with the data given in the folder “hyperelasticity/data/invariants”, where the values of the invariants for the calibration and test datasets are given. Then, implement the potential from Eq. (3) and check your code with the data imported above. Finally, implement the evaluation of the first Piola-Kirchhoff stress

$$\mathbf{P} = \frac{\partial W(\mathbf{F})}{\partial \mathbf{F}} \quad (6)$$

by using JAXs “grad” function, and evaluate it with the data imported above.

2 Neural network model – A naïve approach

2.1 Model implementation

We want to formulate a model which maps the deformation gradient $\mathbf{F}$ to the first Piola-Kirchhoff stress $\mathbf{P}$. A naïve approach to this task is to use a FFNN which has the six independent components of the right Cauchy-Green tensor $\mathbf{C}$ as input and yields the nine components of $\mathbf{P}$ as output. Implement this model, which we denote by M^S ,

As this model uses the right Cauchy-Green tensor as input quantity, it fulfills the objectivity condition. What other conditions does the model fulfill?

2.2 Model calibration

Calibrate the model to the calibration data imported in Sec. 1.1. Is the model able to interpolate the calibration dataset? Is the model able to predict the unseen load cases of the test dataset? Does the model behave physically sensible?

2.3 Loss weighting strategy

The calibration dataset consists of several different load paths. In general, the stress values $\mathbf{P}$ of different load paths may have different orders of magnitude, which affects the contributions of the load paths to the loss function. A load path with smaller stress values will contribute less to the loss function, and thus will be less considered in the parameter optimization process. At worst, the NN won't learn at all from a load path with very small stress values. This can be counteracted by weighting the contributions of different load paths to the loss function: load paths with smaller absolute stress values will be weighted more, while load paths with larger absolute stress values will be weighted less. By this, all different load paths will be equally considered in the parameter calibration process.

Here, we want to use a weighting strategy using (a discrete approximation of) the L^2 norm. For this, implement a function which calculates the norm of each load path according to

$$w = \frac{1}{\#(D)} \sum_j \|\mathbf{P}^j\|, \quad (7)$$

where $\|\cdot\|$ denotes the Frobenius norm, $\#(D)$ denotes the number of elements in the load path, the index j goes through all elements in the load path, and $\mathbf{P}^j$ denotes a single stress tensor of the load path. Use the inverse of w of each load path as its loss weight in the calibration process. Calibrate the model again using the loss weighting strategy, and use the loss weighting strategy for all following model calibrations.

3 Physics-augmented neural network model

3.1 Model implementation

Now, the physics-augmented NN model based on invariants, which we denote by W^I , is to be implemented [4, 7]. The model architecture is as follows:

- The model uses the vector of invariants $\mathcal{I} = (I_1, J, -J, I_4, I_5)$ as input for an ICNN, with the invariants as defined in Eq. (4).
- The scalar-valued output of the ICNN is then used as a hyperelastic potential W .
- Differentiating the ICNN w.r.t. the deformation gradient $\mathbf{F}$ yields the first Piola-Kirchhoff stress $\mathbf{P}$.

Compared to a standard ICNN, the first hidden layer of the ICNN used in this application must be further restricted. How does this restriction look like? Note that this also explains why both J and $-J$ are used in the invariant vector. Hint: Have a look at the polyconvexity condition. Which physical / mathematical conditions are fulfilled by which part of the model architecture?

3.2 Model calibration

Calibrate the model to the calibration data imported in Sec. 1.1.

- Is the model able to interpolate the calibration dataset?
- Is the model able to predict the unseen load cases of the test dataset?
- Examine the stress / energy prediction of the model in the reference configuration $\mathbf{F} = \mathbf{I}$.
- Calibrate the model (a) using only the energy, (b) using only the stress, and (c) using both. For all three cases, evaluate both stress and energy prediction of the model.
- Use different load paths for the calibration of the model, e.g. only uniaxial tension. Is the model still able to extrapolate to the other cases?

4 Concentric sampled deformation gradients – Generalization of NNs

In the previous tasks we observed an excellent generalization behavior for the physics-augmented model, while the naïve approach generalized very poorly. This does *not* imply that the naïve approach is not able to generalize at all. It only shows that with the considered setting, in particular, the model's hyperparameters and the available calibration data, the model showed a poor generalization behavior.

In Sec. 1 deformation paths such as uniaxial tension were used, which are commonly applied in experimental investigations. For this, only a fairly small amount of deformation gradient paths was considered. Now, a calibration dataset consisting of significantly more deformation gradients is used, sampled in a wider range. As uniform sampling of $\mathbf{F}$ in $\mathbb{R}^{3 \times 3}$ would quickly exceed a reasonable dataset size while possibly containing large deformations outside a relevant range, $\mathbf{F}$ is sampled in a physically sensible range using an algorithm proposed by Kunc and Fritzen [6], see [5, Appendix B] for a short introduction.

In “hyperelasticity/data/concentric” you find the deformation gradients generated with this method. Use the analytical model implemented in Sec. 1.2 to generate the corresponding stress tensors. The dataset again consists of different load paths, which are stored in different text files. Out of all load paths, randomly choose different amounts of load paths for the calibration dataset. Calibrate both models to the chosen calibration dataset and use the remaining load paths as test dataset. Increase the amount of load paths in the calibration dataset until both models are able to predict the test dataset. Repeat this investigations five times in order to circumvent the influence of the random choice of load paths for the calibration dataset.

What differences do you observe in the generalization of the models? What difference do you observe in the required network size / calibration dataset size? What is the smallest possible architecture for each model? What is the *inductive bias*, c.f. Haussler [3], of both NN models?

5 Homogenized cubic lattice metamaterials – Approximation of mathematical conditions

5.1 Data preparation

In the first step, data for the calibration and testing of the NN models is to be prepared. Here, synthetic data from homogenized cubic lattice metamaterials will be considered, generated by Fernández et al. [2]. For a short introduction to the dataset, see Klein et al. [4, Section 4.1]. Note that therein the first Piola-Kirchhoff stress is denoted as $\mathbf{S}$, while the deformation gradient is denoted as $\mathbf{F}$. In this tutorial, the first Piola-Kirchhoff stress is denoted as $\mathbf{P}$, while the deformation gradient is denoted as $\mathbf{F}$. Visit the GitHub repository [CPShub/sim-data](#) and download the data for the “BCC” cell. Prepare and visualize the data.

For a stable calibration of NNs it is often convenient to use normalization, e.g., to scale the inputs and outputs of the NN in such a way that they are distributed around 1. For the physics-augmented models considered in this tutorial, it is not possible to normalize

the NNs input, as this would violate the underlying model physics. However, the output (namely the strain energy and the stress tensor) can be scaled according to

$$W^* = a W(\mathbf{F}), \quad \mathbf{P}^* = a \mathbf{P}, \quad a \geq 0, \quad (8)$$

where W denotes the original strain energy and $\mathbf{P}$ denotes the original stress tensor. Scale the data with a suitable factor a so that the components of $\mathbf{P}^*$ are in the order of 1.

- Why is it not possible to use batch normalization for this model?
- What are the benefits of scaling the output as in eq. (8)?

5.2 Invariant-based model

In Sec. 3 the physics-augmented NN model W^I based on invariants was implemented for transversely isotropic materials. Adapt the model for cubic anisotropy by using the vector of invariants

$$\mathcal{I} = (I_1, I_2, J, -J, I_7, I_{11}) \in \mathbb{R}^6 \quad (9)$$

as input for the model, with the invariants

$$I_1 = \text{tr } \mathbf{C}, \quad I_2 = \text{tr}(\text{cof } \mathbf{C}) \quad J = \det \mathbf{F}, \quad I_7 = \mathbf{C} : \mathbb{G}_{\text{cub}} : \mathbf{C}, \quad I_{11} = \text{cof } \mathbf{C} : \mathbb{G}_{\text{cub}} : \text{cof } \mathbf{C} \quad (10)$$

and the fourth order cubic structural tensor

$$\mathbb{G}_{\text{cub}} = \sum_{i=1}^3 \mathbf{e}_i \otimes \mathbf{e}_i \otimes \mathbf{e}_i \otimes \mathbf{e}_i, \quad (11)$$

with $\mathbf{e}_i$ being the basis vectors of a Cartesian coordinate system. Use the uniaxial and shear case for the calibration dataset, and the mixed shear-tension case “test 3” for the test dataset.

- Is the model able to interpolate the dataset? What could be the challenge with this dataset?

5.3 Deformation-gradient based neural network model

Consider the definition of polyconvexity: The potential $W(\mathbf{F})$ is polyconvex if and only if there exists a function $\mathcal{P} : \mathbb{R}^{3 \times 3} \times \mathbb{R}^{3 \times 3} \times \mathbb{R} \rightarrow \mathbb{R}$ such that

$$W(\mathbf{F}) = \mathcal{P}(\mathbf{F}, \text{cof } \mathbf{F}, \det \mathbf{F}), \quad (12)$$

so that $\mathcal{P}$ is convex in its arguments [1]. If we use the components of $(\mathbf{F}, \text{cof } \mathbf{F}, \det \mathbf{F}) \in \mathbb{R}^{19}$ as inputs for an ICNN, the scalar-valued output of the ICNN will be polyconvex by construction. However, as we don't use invariants anymore, neither the objectivity condition nor the material symmetry condition are fulfilled by construction. Thus, further steps are required to take both conditions into account. For this task, only the objectivity condition is considered. In hyperelasticity, the objectivity condition is given by

$$W(\mathbf{QF}) = W(\mathbf{F}) \quad \forall \mathbf{F} \in \text{GL}^+(3), \mathbf{Q} \in \text{SO}(3), \quad (13)$$

which implies the stress invariance condition

$$\mathbf{P}(\mathbf{QF}) = \mathbf{Q} \mathbf{P}(\mathbf{F}) \quad \forall \mathbf{F} \in \text{GL}^+(3), \mathbf{Q} \in \text{SO}(3). \quad (14)$$

Here,

$$\text{GL}^+(3) := \{\mathbf{X} \in \mathbb{R}^{3 \times 3} | \det \mathbf{X} > 0\} \quad (15)$$

denotes the set of invertible second order tensors with positive determinant, while

$$\text{SO}(3) := \{\mathbf{X} \in \mathbb{R}^{3 \times 3} | \mathbf{X} \mathbf{X}^T = \mathbf{I}, \det \mathbf{X} = 1\} \quad (16)$$

denotes the special orthogonal group in $\mathbb{R}^3$. Simply said, the $\text{SO}(3)$ contains all rotation matrices in the three-dimensional space without inversion.

Adapt the model of Sec. 3 so that it takes the components of $(\mathbf{F}, \text{cof } \mathbf{F}, \det \mathbf{F}) \in \mathbb{R}^{19}$ as input. We denote this model as W^F . Calibrate the model using the calibration dataset defined in Sec. 5.2. Implement an evaluation of W^F which evaluates the model for multiple observers. For the multiple observers, choose a finite amount of randomly distributed rotation matrices. Check this evaluation using the invariant-based model W^I . Then, for the calibrated model W^F , evaluate the invariance condition on both the calibration and test dataset and find a suitable visualization for the results of the different observers.

- How does the polyconvexity condition restrict the first hidden layer of the NN when the components of $(\mathbf{F}, \text{cof } \mathbf{F}, \det \mathbf{F}) \in \mathbb{R}^{19}$ are used as inputs? What is the difference to the invariant-based PANN model, see Sec. 3?
- Which physical / mathematical conditions are fulfilled by this model?

5.4 Data augmentation

The model W^F does not fulfill the objectivity condition by construction. Now, data augmentation is used to “teach” the model how to fulfill this condition in an approximate fashion. For the data augmentation, based on the initial dataset $D = \{F, W, P\}$, the dataset is extended by a finite amount of artificial observers according to

$$\tilde{D} = \bigcup_{Q_{\text{obj}} \in \mathcal{G}_{\text{obj}}} \{Q_{\text{obj}} F, W, Q_{\text{obj}} P\}. \quad (17)$$

Here, $\mathcal{G}_{\text{obj}} \subset \text{SO}(3)$ denotes a finite amount of randomly distributed rotation matrices. Augment the initial calibration dataset and calibrate the model W^F . Increase the number of rotation matrices included in $\mathcal{G}_{\text{obj}}$ in [8, 16, 32, ...] until the objectivity condition is approximated “well enough”.

- How must the test dataset be chosen to check the fulfillment of the objectivity condition?
- Out of all datasets and models you used in this tutorial – which one would you prefer in each case?

References

- [1] V. Ebbing. “Design of Polyconvex Energy Functions for All Anisotropy Classes”. PhD thesis. Universität Duisburg-Essen, 2010.
- [2] M. Fernández, M. Jamshidian, T. Böhlke, K. Kersting and O. Weeger. “Anisotropic hyperelastic constitutive models for finite deformations combining material theory and data-driven approaches with application to cubic lattice metamaterials”. In: *Computational Mechanics* 67.2 (2021), pp. 653–677. doi: 10.1007/s00466-020-01954-7.
- [3] D. Haussler. “Quantifying inductive bias: AI learning algorithms and Valiant’s learning framework”. In: *Artificial Intelligence* 36.2 (1988), pp. 177–221. doi: 10.1016/0004-3702(88)90002-1.
- [4] D. K. Klein, M. Fernández, R. J. Martin, P. Neff and O. Weeger. “Polyconvex anisotropic hyperelasticity with neural networks”. In: *Journal of the Mechanics and Physics of Solids* 159 (2022), p. 104703. doi: 10.1016/j.jmps.2021.104703.
- [5] D. K. Klein, R. Ortigosa, J. Martínez-Frutos and O. Weeger. “Finite electro-elasticity with physics-augmented neural networks”. In: *Computer Methods in Applied Mechanics and Engineering* 400 (2022), p. 115501. arXiv: 2206.05139.
- [6] O. Kunc and F. Fritzen. “Finite strain homogenization using a reduced basis and efficient sampling”. In: *Mathematical and Computational Applications* 24.2 (2019), p. 56. doi: 10.3390/mca24020056.
- [7] L. Linden, D. K. Klein, K. A. Kalina, J. Brummund, O. Weeger and M. Kästner. “Neural networks meet elasticity: A guide for enforcing physics”. In: *Journal of the Mechanics and Physics of Solids* 179 (2023), p. 105363. doi: 10.1016/j.jmps.2023.105363.
- [8] J. Schröder, P. Neff and V. Ebbing. “Anisotropic polyconvex energies on the basis of crystallographic motivated structural tensors”. In: *Journal of the Mechanics and Physics of Solids* 56 (2008), pp. 3486–3506. doi: 10.1016/j.jmps.2008.08.008.